МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «ЛИПЕЦКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Институт (факультет) Институт компьютерных наук
Кафедра Автоматизированные системы управления

ЛАБОРАТОРНАЯ РАБОТА № 2
Реализация экспертной системы с использованием обратной цепочки рассуждений
По дисциплине: «Системы искусственного интеллекта»

Студент ПИ-23-2 Домашнев А.П.
Шадров И.А

группа | подпись, дата | фамилия, инициалы

Руководитель

Первушин О.С.

ученая степень, ученое звание | подпись, дата | фамилия, инициалы

Липецк 2026

1. Задание кафедры

1. Разработайте оболочку экспертной системы, реализующую обратную цепочку рассуждений, на любом языке программирования.

2. Требования к базе знаний аналогичны требованиям в лабораторной работе № 1.

3. Задайте начальное состояние рабочей базы данных.

4. Задайте целевую ситуацию.

5. С помощью обратной цепочки рассуждений докажите или опровергните возможность достижения заданной целевой ситуации.

6. Выберите предметную область из лабораторной работы № 1 или предложите другую, для которой составьте базу правил.

7. Протестируйте работу вашей программы на этой базе правил.

2. Описание алгоритма

Хранение промежуточных данных в данном варианте использования требует реализации древовидной структуры из-за того, что для проверки достижимости требуется найти такой набор входных состояний, который приведет к конечному состоянию.

Весь алгоритм состоит из построения этого дерева и проверки существования допустимой комбинации.

Для построения дерева требуется пройти по всем правилам в базе правил и для каждого правила проверить, существует ли в текущем банке переменная, являющаяся заключением. Если существует, то нужно добавить в качестве узлов текущего узла переменные, от которых зависит найденная переменная.

Для проверки существования допустимой комбинации требуется найти такое поддерево с высотой больше 1, листья которого представлены в заданном начальном состоянии базы данных.

3. Фрагменты кода

В приложении А представлены фрагменты кода, реализующие алгоритм расчетов. Для реализации был выбран язык C# и фреймворк WPF [1].

4. Пример работы

Ниже представлено содержимое базы данных правил.

(состояние = общее недомогание) => (головная боль = да) И (слабость = да)

(состояние = общее недомогание) => (температура = высокая) И (слабость = да) И (потливость = да)

(состояние = лихорадка) => (температура = высокая) И (озноб = да)

(состояние = кардиальный синдром) => (боль в груди = да) И (одышка = да) И (учащённое сердцебиение = да)

(состояние = кардиальный синдром) => (боль в груди = да) И (отёки конечностей = да)

(состояние = неврологический синдром) => (головная боль = да) И (тошнота = да) И (светочувствительность = да)

(состояние = неврологический синдром) => (онемение конечностей = да) И (головокружение = да) И (нарушение координации = да)

(состояние = опорно-двигательный синдром) => (боль в суставах = да) И (отёк сустава = да) И (ограничение подвижности = да)

(состояние = опорно-двигательный синдром) => (боль в спине = да) И (скованность движения = да) И (онемение спины = да)

(состояние = лор-инфекция) => (боль в горле = да) И (насморк = да) И (температура = высокая)

(состояние = лор-инфекция) => (снижение слуха = да) И (боль в ухе = да) И (выделения из уха = да)

(состояние = глазная патология) => (боль в глазах = да) И (снижение зрения = да) И (покраснение глаз = да)

(состояние = глазная патология) => (слезотечение = да) И (светочувствительность = да) И (снижение зрения = да)

(состояние = дерматит) => (сыпь на коже = да) И (зуд = да) И (покраснение кожи = да)

(состояние = дерматит) => (шелушение кожи = да) И (сухость кожи = да) И (зуд = да)

(состояние = гастроэнтерит) => (боль в животе = да) И (тошнота = да) И (нарушение стула = да)

(состояние = гастроэнтерит) => (изжога = да) И (тошнота = да) И (вздутие живота = да)

(состояние = пневмония) => (кашель = да) И (одышка = да) И (боль в грудной клетке = да)

(состояние = бронхит) => (кашель = да) И (выделения мокроты = да) И (температура = высокая)

(состояние = аллергическая реакция) => (сыпь на коже = да) И (отёк = да) И (затруднённое дыхание = да)

(состояние = стоматологическая патология) => (боль в зубах = да) И (отёк дёсен = да) И (кровоточивость дёсен = да)

(специальность = терапевт) => (состояние = общее недомогание)

(специальность = терапевт) => (состояние = лихорадка)

(специальность = кардиолог) => (состояние = кардиальный синдром)

(специальность = невролог) => (состояние = неврологический синдром)

(специальность = ортопед) => (состояние = опорно-двигательный синдром)

(специальность = лор) => (состояние = лор-инфекция)

(специальность = офтальмолог) => (состояние = глазная патология)

(специальность = дерматолог) => (состояние = дерматит)

(специальность = гастроэнтеролог) => (состояние = гастроэнтерит)

(специальность = пульмонолог) => (состояние = пневмония)

(специальность = пульмонолог) => (состояние = бронхит)

(специальность = аллерголог) => (состояние = аллергическая реакция)

(специальность = стоматолог) => (состояние = стоматологическая патология)

На рисунке 1 представлен пример результата выполнения алгоритма.

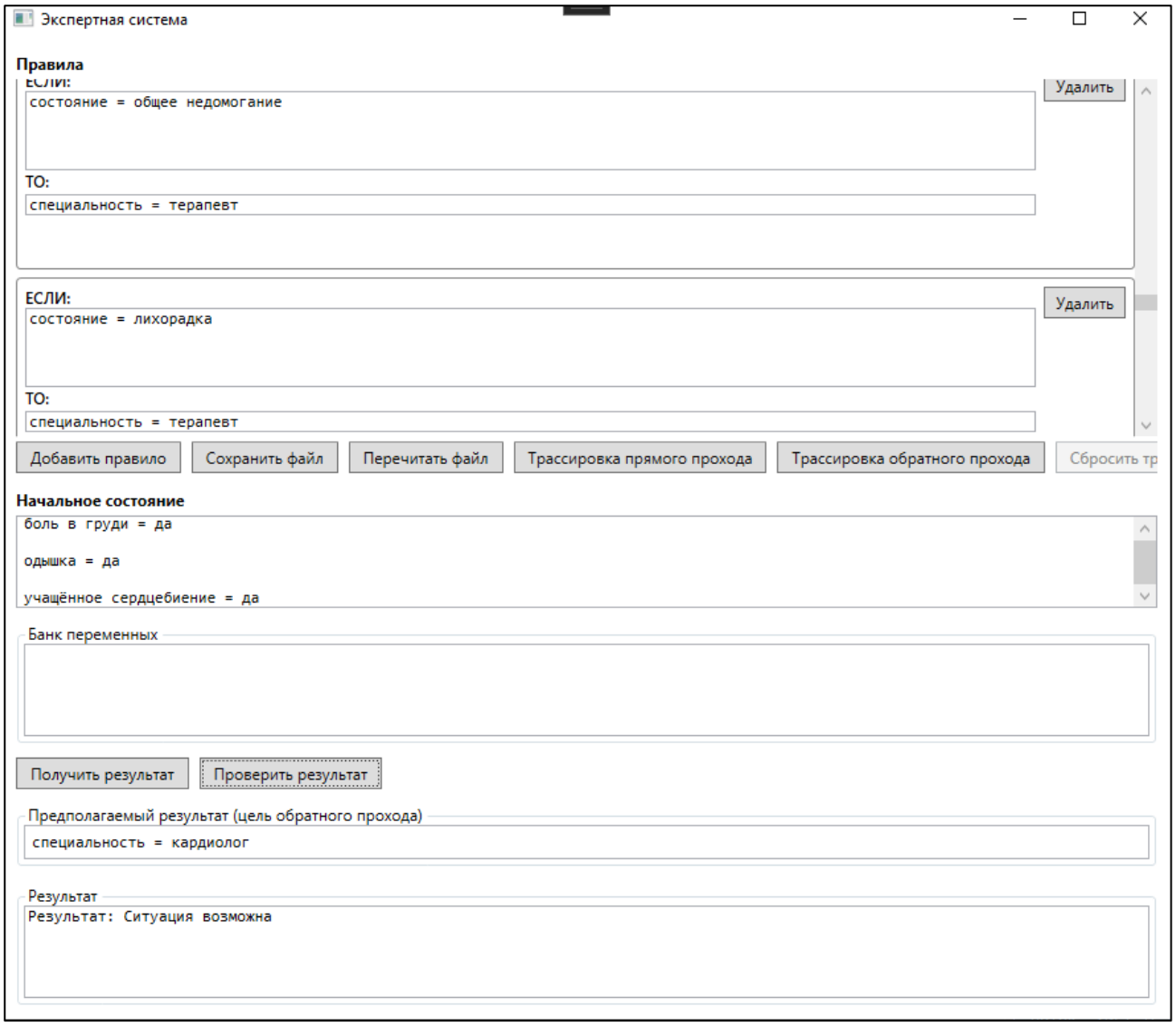


Рисунок 1 – Результат выполнения

Выводы

В процессе работы были изучены методы реализации экспертных систем с использованием прямой цепочки рассуждений.

Список литературы

1. Microsoft [Электронный ресурс] Windows Presentation Foundation | Microsoft Learn.- режим доступа: https://learn.microsoft.com/ru-ru/dotnet/desktop/wpf/get-started/create-app-visualstudio?view=netdesktop-9.0

Приложение А

(Обязательное)

1. ExpertSystem.cs

```
namespace AIS.Math
{
    internal class ExpertSystemVerdict
    {
        public bool IsSuccess { get; set; }
        public required string Result { get; init; }
    }

    internal class ExpertSystem
    {
        public string Subsymbols { get; set; } = ",.;!?1234567890+:-/";

        private List<ExpertSystemToken> _tokens = new List<ExpertSystemToken>();

        public ExpertSystem()
        {

        }

        public ExpertSystemToken ParseToken(string text)
        {
            text = text.Trim();

            foreach (var sym in Subsymbols.Select(c => c.ToString()))
            {
                text = text.Replace(sym, "");
            }

            ExpertSystemToken newToken = new ExpertSystemToken()
            {
                Object = text
                    .Split("=", StringSplitOptions.TrimEntries |
StringSplitOptions.RemoveEmptyEntries)
                    .First(),
                Value = text
                    .Split("=", StringSplitOptions.TrimEntries |
StringSplitOptions.RemoveEmptyEntries)
                    .ElementAtOrDefault(1)
                    ?? "Да",
            };

            return newToken;
        }

        public void AddToken(string text)
        {
            _tokens.Add(ParseToken(text));
        }

        public ExpertSystemVerdict GetResult()
        {
            ConditionsContext context = ConditionsContext.Get();

            bool flag = true;
            List<ExpertSystemToken> tokens = [.._tokens];
            List<ConditionSequence> conditions = [..context.ConditionSequences];

            while (flag)
            {
                flag = false;
                for (int i = 0; i < conditions.Count; i++)
                {
                    LogContext.AddConditionInspectState(conditions[i]);
                    if (conditions[i].TryFit(tokens, out ConditionItem newItem))
```

```
                    {
                        LogContext.AddCommitState(conditions[i]);
                        conditions.RemoveAt(i);
                        tokens.Add(new ExpertSystemToken() { Object = newItem.Object, Value
=  newItem.Value });
                        i--;
                        flag = true;
                        LogContext.AddVariableState(new ExpertSystemToken() { Object =
newItem.Object, Value = newItem.Value });
                    }
                }
            }

            if (tokens.FirstOrDefault(t => t.Object.Equals("специальность",
StringComparison.OrdinalIgnoreCase)) is not null and var result)
            {
                return new ExpertSystemVerdict() { Result = result.Value, IsSuccess = true
};
            }
            else
            {
                return new ExpertSystemVerdict() { Result = "Предоставьте дополнительные
факты.", IsSuccess = false };
            }
        }

        public ExpertSystemVerdict GetRevesedResult(ExpertSystemToken result)
        {
            ConditionsContext context = ConditionsContext.Get();

            bool flag = true;
            List<ExpertSystemToken> tokens = [.. _tokens];

            TokenBankTree bank = new() { Parent = null };
            bank.AddToken(result);

            List<ConditionSequence> conditions = [.. context.ConditionSequences];

            LogContext.AddGoalSetState(result);

            int iteration = 0;

            while (flag)
            {
                flag = false;
                iteration++;

                for (int i = 0; i < conditions.Count; i++)
                {
                    LogContext.AddReverseConditionInspectState(conditions[i], iteration);

                    foreach (var token in bank.GetCurrentTokens())
                    {
                        if (conditions[i].ReverseFit(token, out List<ConditionItem> items))
                        {
                            LogContext.AddReverseCommitState(conditions[i]);

                            foreach (var item in items)
                            {
                                var required = new ExpertSystemToken() { Object =
item.Object, Value = item.Value };
                                LogContext.AddBankAddState(item, required);
                                bank.Add(token, required);
                            }

                            flag = true;
                            conditions.RemoveAt(i);
                            i--;
                        }
                    }
                }
```

```
            }

            bool verified = false;

            if (bank.GetLeafs().Count() == 0)
            {
                verified = false;
            }

            var bankState = bank.GetLeafs().ToList();

            while (bankState.Any(bs => bs != null))
            {
                for (int i = 0; i < bankState.Count(); i++)
                {
                    if (bankState[i] == null) continue;

                    if (bankState[i].Inner.Keys.All(k => tokens.Any(t => t.Equals(k))))
                    {
                        verified = true;
                        goto ver;
                    }
                    else
                    {
                        bankState[i] = bankState[i].Parent;
                    }
                }
            }

        ver: if (verified)
            {
                return new ExpertSystemVerdict() { Result = "Ситуация возможна", IsSuccess
= true };
            }
            else
            {
                return new ExpertSystemVerdict() { Result = "Ситуация недостижима",
IsSuccess = true };
            }
        }
    }
}
```

2. ConditionSequence.cs

```
namespace AIS.Model
{

    internal class ConditionSequence
    {
        public List<ConditionItem> Input { get; set; } = [];
        public required ConditionItem Output { get; init; }

        public bool TryFit(List<ExpertSystemToken> input, out ConditionItem newItem)
        {
            for (int begin = 0; begin < input.Count; begin++)
            {
                foreach (var item in Input.Where(i => !i.IsChecked))
                {
                    LogContext.AddInspectState(item, input[begin]);
                    if (item.Fit(input[begin]))
                    {
                        LogContext.AddPartialState(item);
                    }
                }
            }

            bool result = Input.All(i => i.IsChecked);

            if (result)
            {
                newItem = Output;
```

```
            }
            else
            {
                newItem = null!;
            }

            return result;
        }

        public bool ReverseFit(ExpertSystemToken output, out List<ConditionItem>
dependencies)
        {
            LogContext.AddReverseInspectState(Output, output);
            if (Output.Fit(output))
            {
                dependencies = Input;
                return true;
            }
            dependencies = null!;
            return false;
        }
    }
}
```

3. ConditionItem.cs

```
namespace AIS.Model
{
    internal class ConditionItem
    {
        public required string Object { get; init; }
        public required string Value { get; init; }

        public bool IsChecked { get; set; } = false;

        public bool Fit(ExpertSystemToken input)
        {
            if (IsChecked is true) return true;

            return IsChecked = Object.Equals(input.Object,
StringComparison.OrdinalIgnoreCase) && Value.Equals(input.Value,
StringComparison.OrdinalIgnoreCase);
        }
    }
}
```

4. ExpertSystemToken

```
namespace AIS.Model
{
    internal class ExpertSystemToken
    {
        public string Object { get; set; }
        public string Value { get; set; }
    }
}
```

5. TokenBankTree

```
namespace AIS.Model
{
    internal class TokenBankTree
    {
        public Dictionary<ExpertSystemToken, TokenBankTree> Inner { get; set; } = new();
        public required TokenBankTree? Parent { get; set; }

        public List<ExpertSystemToken> GetCurrentTokens()
        {
            return GetLeafs().SelectMany(l => l.Inner.Keys).ToList();
        }

        internal void Add(ExpertSystemToken token, ExpertSystemToken required)
```

```
        {
            if (Inner.Keys.Any(k => k.Equals(token)))
            {
                Inner[token].AddToken(required);
            }
            else
            {
                foreach (var inner in Inner.Keys)
                {
                    Inner[inner].Add(token, required);
                }
            }
        }

        internal void AddToken(ExpertSystemToken token)
        {
            Inner.Add(token, new() { Parent = this });
        }

        internal IEnumerable<TokenBankTree> GetLeafs()
        {
            if (Inner.Values.All(i => i.Inner.Count == 0)) return [this];

            return Inner.SelectMany(i => i.Value.GetLeafs());
        }
    }
}
```